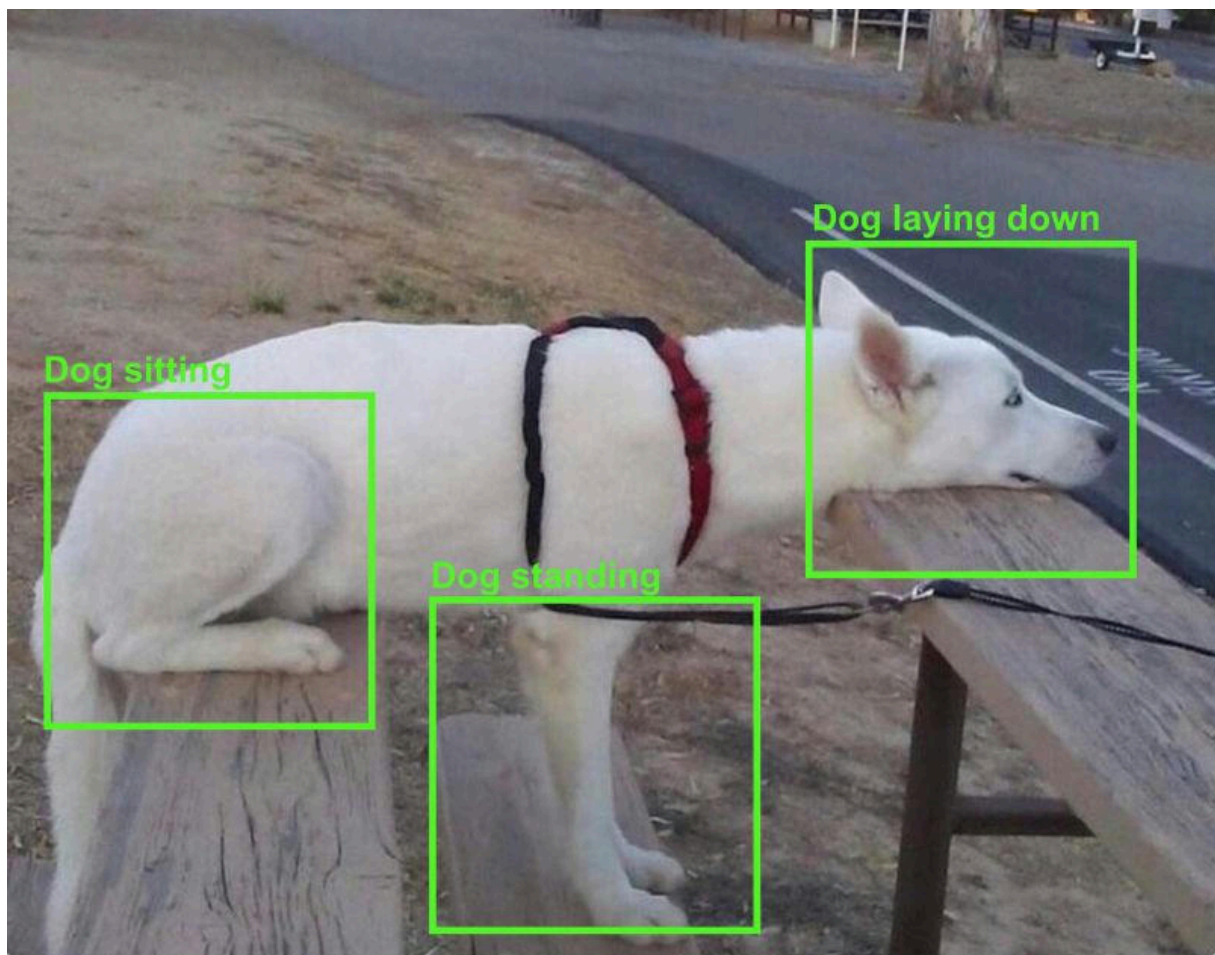


Memoria Práctica 3

Aprendizaje Automático

Jorge Alcalde Vesteiro
Andy Barcia Rodríguez
Pedro Pablo López Mena



Índice

1) Introducción	3
2) VISIÓN	3
2.a) Evaluación	3
2.b) Perceptrón multicapa	4
2.b.a) Baseline Scikit Learn	4
2.b.b) Mejoras	5
2.b.c) Tensorflow y Keras	8
2.c) Red convolucional	10
2.c.a) Baseline	10
2.c.b) Mejoras	13
2.d) Conclusión	16
3) NLP	16
3.a) Evaluación	17
3.b) Preprocesamiento	17
3.c) Resultados	17
3.c.a) Baseline	18
3.c.a.a) Código	19
3.c.a.b) Resultados	20
3.c.b) Mejoras	20
3.c.b.a) Configuración del tokenizador	23
3.c.b.a.a) Código	23
3.c.b.a.b) Resultados	24
3.c.b.b) Capas recurrentes, tipos y tamaños	24
3.c.b.b.a) Código	24
3.c.b.b.b) Resultados	25
3.c.b.c) Otras capas y Dropout	25
3.c.b.c.a) Código	26
3.c.b.c.b) Resultados	26
3.c.b.d) Otros optimizadores	26
3.c.b.d.a) Código	27
3.c.b.d.b) Resultados	28
3.c.c) Mejor Modelo	28
3.c.c.a) Código	29
3.c.c.b) Resultados	31
3.c.d) Otras mejoras	31
3.c.d.a) Código	31
3.c.d.b) Resultados	34
3.c.e) CONCLUSIÓN	34

1) Introducción

En el siguiente trabajo realizaremos una familiarización con tres tipos diferentes de redes neuronales, los perceptrones multicapa, las redes convolucionales y redes recurrentes. Para realizar lo anterior se resolverán dos problemas distintos, por un lado la clasificación y predicción de categorías de ropa a través de visión computacional, por el otro lado el análisis de sentimientos de reviews de productos en amazon.

Para ambos problemas se busca la implementación de los algoritmos manifestados anteriormente, primero en un análisis que nos permita comprender las ventajas y desventajas de cada uno de los algoritmos, por ende inicialmente sin utilizar los parametros de ajuste, y luego haciendo una mejora a tales implementaciones en virtud de la comprensión de los parametros aplicados al problema.

Para el trabajo se utilizarán las librerías skitlearn, tensorflow y keros, haciendo un trabajo similar con cada una de ellas.

2) VISIÓN

En este apartado vamos a centrarnos en modelos aplicados en visión artificial, más específicamente, entrenados en el *dataset* fashion MNIST. Esta es una versión más difícil del popular MNIST convencional de reconocimiento de dígitos, ya que dicho *dataset* es muy sencillo de aprender para las arquitecturas modernas de visión artificial. El *dataset* cuenta con 70,000 imágenes de 28×28 píxeles en escala de grises, equitativamente divididos en 10 categorías.

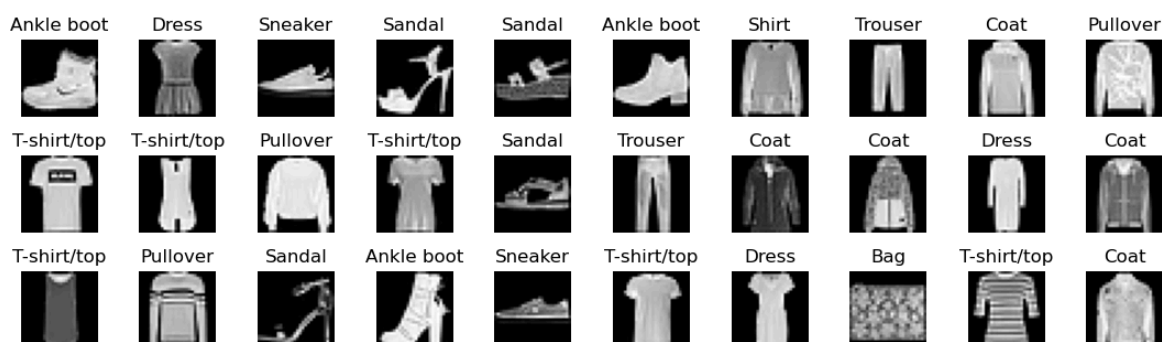


Figura 1: Algunas imágenes de entrenamiento del *dataset* fashion MNIST.

El *dataset* se cargará directamente con la API de Keras, que tiene éste y muchos otros *datasets* para trabajar directamente.

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

x_train = tf.convert_to_tensor(x_train)
y_train = tf.convert_to_tensor(y_train)

x_test = tf.convert_to_tensor(x_test)
y_test = tf.convert_to_tensor(y_test)
```

2.a) Evaluación

Como el *dataset* está perfectamente balanceado, se ha considerado que la mejor opción de evaluación es el accuracy sobre el *dataset* de test de fashion MNIST. Esto permitirá comparar de forma justa gran cantidad de modelos con distintas arquitecturas.

2.b) Perceptrón multicapa

Utilizar un perceptrón multicapa para procesar cada pixel de una imagen es una técnica poco utilizada hoy en día debido a la cantidad de recursos que requiere para una imagen de un tamaño considerable. Pero ya que estamos trabajando con imágenes bastante pequeñas, es un problema abordable. En este caso lo que se hará para todas las imágenes es convertirlas del rango (0, 255) al rango (0, 1), y colapsando las matrices de tamaño 28×28 a vectores de longitud 784.

```
def preprocess(imgs : np.array, collapse=True) -> np.array:
    imgs = tf.cast(imgs, tf.float32) / 255.0
    if collapse:
        imgs = rearrange(imgs, "b h w -> b (h w)")
    return imgs
```

2.b.a) Baseline Scikit Learn

El entrenamiento de un perceptrón multicapa con Scikit es muy sencillo. El código de preprocesamiento de los datos, entrenamiento y validación lo vamos a incorporar todo en una misma función personalizable para así comparar fácilmente distintos modelos.

```
from sklearn.neural_network import MLPClassifier

def train_sklearn_mlp(preprocess=preprocess, **kwargs):
    model = MLPClassifier(**kwargs)

    # Training
    if preprocess:
        imgs = preprocess(x_train)
    else:
        imgs = x_train
    model.fit(imgs, y_train)

    # Eval
    if preprocess:
        imgs = preprocess(x_test)
    else:
        imgs = x_test
    predict_labels = model.predict(imgs)
    score = sklearn.metrics.accuracy_score(y_test, predict_labels)

    return model, score
```

El entrenamiento se ha realizado con la siguiente configuración, utilizando solo 16 neuronas en la capa oculta, y un learning rate de 0,001. Se utiliza un random_state inicial para garantizar la reproducibilidad de los resultados en cada ejecución del modelo. Esto es crucial para asegurar que los experimentos sean consistentes y comparables entre sí, permitiendo una evaluación precisa del rendimiento del modelo bajo condiciones controladas.

```
mlp_config = {
    'hidden_layer_sizes': [16],
    'learning_rate_init': 0.001,
    'max_iter': 300,
    'random_state': 25,
```

```

    'batch_size':200,
    'tol': 0.0001,
}

```

Como se puede ver, el modelo va entrenando progresivamente con las épocas, por lo que podemos deducir que el código funciona correctamente. Nótese que parece que el modelo podría seguir entrenando durante bastantes más épocas antes de parar de aprender, pero la implementación de Sklearn es bastante lenta, por lo que no era viable probarlo durante demasiadas épocas.

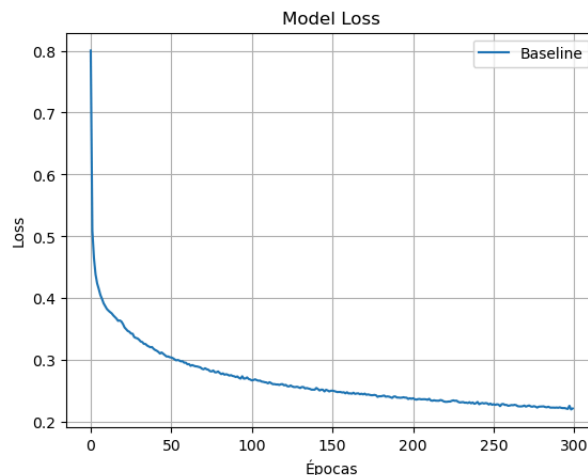


Figura 2: Histórico del loss de entrenamiento a medida que avanza el entrenamiento.

En la última época el modelo alcanza un *accuracy* del 85,71% en el *dataset* de test. Un resultado bastante razonable para lo pequeño que es el modelo.

2.b.b) Mejoras

A continuación se describen algunas de las mejoras que se han probado:

- **Normalización del *dataset*:** esta es una mejora bastante sencilla que consiste en hacer que los píxeles del *dataset* tengan una media de 0 y desviación típica de 1, ayudando a la red a converger más rápidamente. Para ello es tan sencillo como coger la media del *dataset*:

```

means = tf.reduce_mean(tf.cast(x_train, tf.float32)) / 255.0
stds = tf.math.reduce_std(tf.cast(x_train, tf.float32)) / 255.0

```

Y, a continuación, utilizarlos en la función de preprocesado.

```

def preprocess_norm(imgs : np.array, collapse=True) -> np.array:
    imgs = tf.cast(imgs, tf.float32) / 255.0
    imgs = (imgs - means) / stds
    if collapse:
        imgs = rearrange(imgs, "b h w -> b (h w)")
    return imgs

```

- **Reducción de dimensionalidad con PCA:** esta es una técnica que consiste en reducir la dimensionalidad de las entradas al perceptrón (actualmente tienen 784 dimensiones) bajo la suposición de que muchas de las dimensiones son *codependientes*, esto es, que sus valores están altamente correlacionados. Eliminando dimensiones *codependientes* se puede facilitar la convergencia del modelo y posiblemente mejorar sus resultados en el *dataset* de test.

Para elegir el número de dimensiones del PCA podemos utilizar una gráfica que nos muestre la cantidad de varianza del *dataset* que podemos explicar para un número dado de dimensiones del PCA. Como se muestra en la Figura 3, en general parece que las dimensiones no son muy codependientes, pues se necesitan casi 100 dimensiones para explicar un 90% de la varianza. Como queremos que el modelo no se vea muy penalizado, acabamos eligiendo 188 dimensiones, que explican el 95% de la varianza acumulada, y siguen siendo significativamente menos dimensiones que las 784 originales.

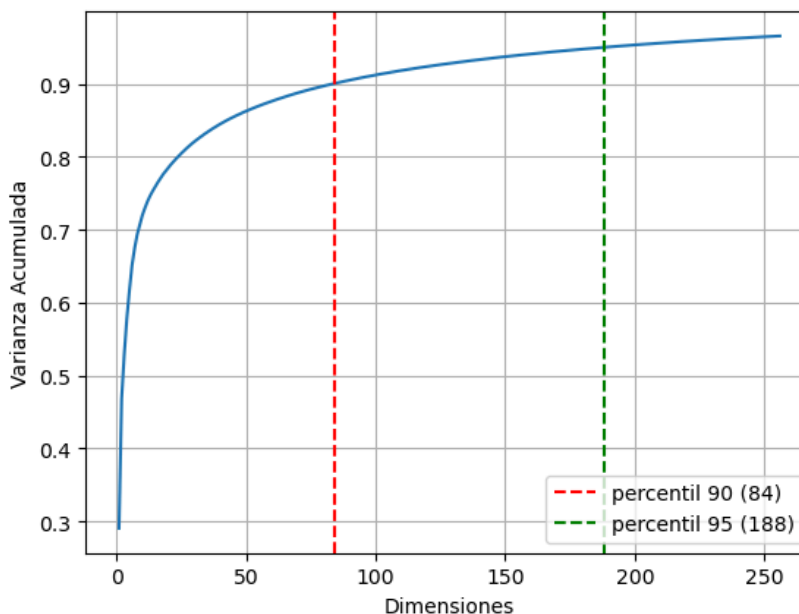


Figura 3: Varianza explicada en base al número de dimensiones del PCA.

Se muestra a continuación el código utilizado. Nótese que antes del PCA realizamos una normalización de los datos, como en el apartado anterior.

```
pca_188 = sklearn.decomposition.PCA(n_components=188)
_ = pca_188.fit(preprocess_norm(x_train))

def preprocess_pca(imgs : np.array, pca=pca_188, collapse=True) -> np.array:
    imgs = tf.cast(imgs, tf.float32) / 255.0
    imgs = (imgs - means) / stds
    if collapse:
        imgs = rearrange(imgs, "b h w -> b (h w)")
    return pca.transform(imgs)
```

- **Reducción de dimensionalidad con LDA:** esta es una técnica similar al PCA, salvo que el número de dimensiones se reduce al número de clases de clasificación (es decir, como tenemos 10 clases, se reduce de 784 a 10 dimensiones), y de tal forma que las distintas clases son lo más linealmente separables posibles. En este caso el código es bastante sencillo:

```
lda = sklearn.discriminant_analysis.LinearDiscriminantAnalysis()
_ = lda.fit(preprocess_norm(x_train), y_train)

def preprocess_lda(imgs : np.array, lda=lda, collapse=True) -> np.array:
    imgs = tf.cast(imgs, tf.float32) / 255.0
    imgs = (imgs - means) / stds
    if collapse:
        imgs = rearrange(imgs, "b h w -> b (h w)")
    return lda.transform(imgs)
```

```

imgs = rearrange(imgs, "b h w -> b (h w)")
return lda.transform(imgs)

```

Para comprobar cómo de linealmente separables han quedado las clases, podemos mostrar gráficamente el *dataset* comparando pares de dimensiones. Como se puede ver en la Figura 4, en general los pares de clases no son *muy* linealmente separables, aunque puede que sí que lo sean en el espacio de 10 dimensiones.

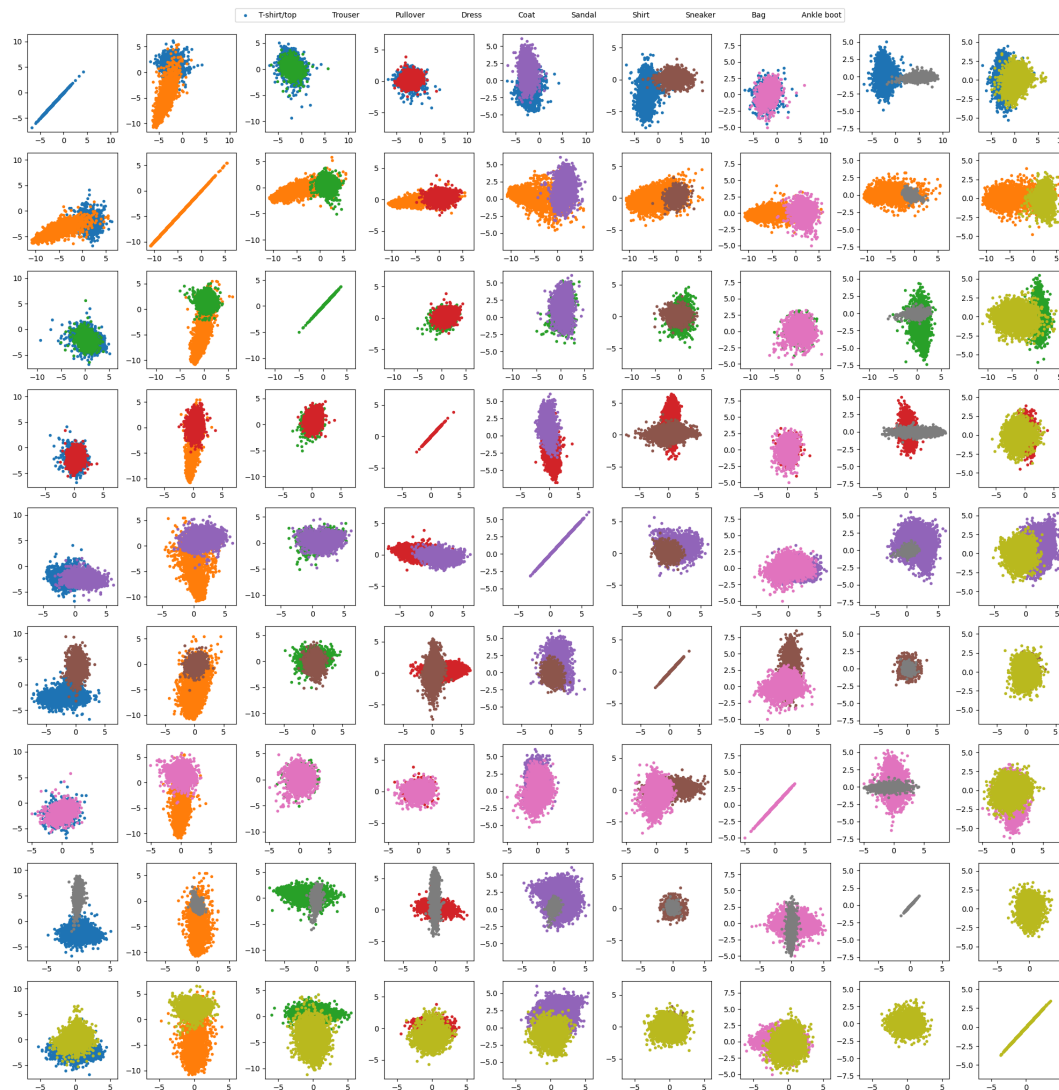


Figura 4: Representación gráfica de los datos del *dataset* con sus 10 dimensiones.

- **Detección de bordes con OpenCV:** por último, vamos a utilizar funciones de OpenCV para procesar el *dataset* de entrenamiento. Más específicamente, hemos probado a utilizar la función de detección de bordes para convertir las imágenes de una escala de grises continua a imágenes binarias (o «borde» o «no borde»).

```

import cv2 as cv

def preprocess_canny(imgs:np.array, low=100, high=200, collapse=True) -> np.array:
    imgs = tf.map_fn(lambda img: cv.Canny(img.numpy(), low, high), imgs)
    imgs = tf.cast(imgs, tf.float32) / 255.0
    if collapse:

```

```

    imgs = rearrange(imgs, "b h w -> b (h w)")
    return imgs

```

En la Figura 5 se muestran unos ejemplos de cómo se ven las imágenes una vez procesadas. Se puede ver que en general, las imágenes no son tan reconocibles como antes, por lo que no esperamos muy buenos resultados de esta mejora.

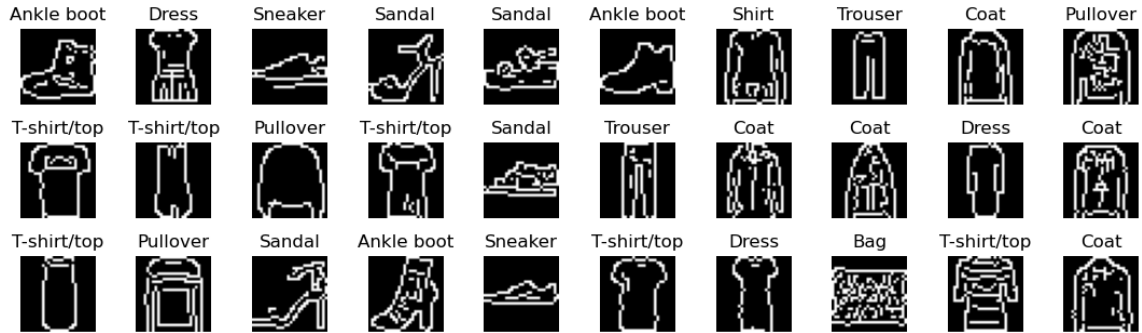


Figura 5: Algunas imágenes de entrenamiento del *dataset* fashion MNIST con preprocesamiento para detección de bordes con OpenCV.

A continuación se muestran los resultados de curvas de *loss* para todos los modelos, y los *accuracies* de test finales. Como se puede ver, aunque la mejora con normalización es quien consigue menos *loss* más rápidamente, en verdad consigue un *accuracy* similar a la implementación normal en el *dataset* de test. El mejor modelo parece ser el PCA de 188 dimensiones, y los peores el LDA y el preprocesamiento con bordes (como habíamos predicho).

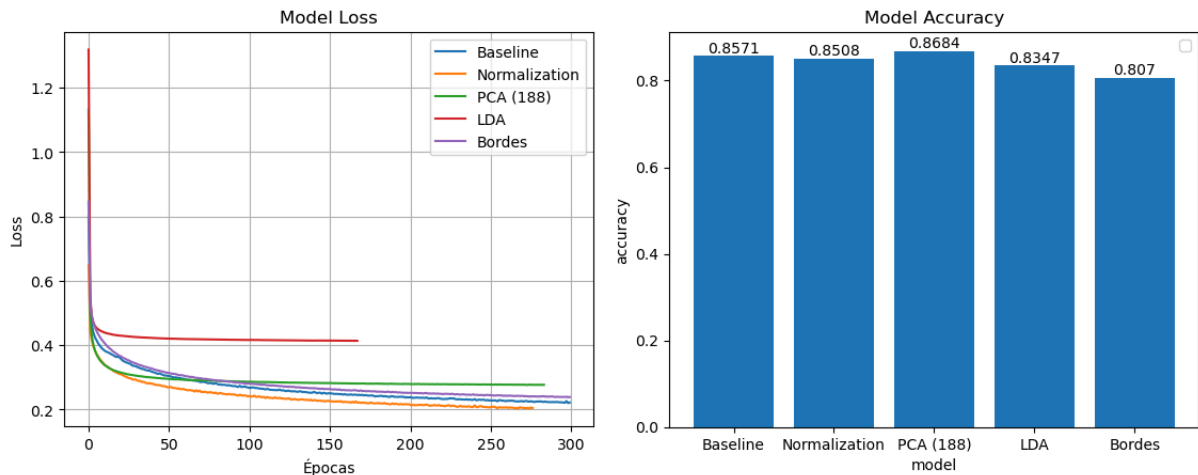


Figura 6: Resultados de *loss* y *accuracy* de todos los modelos probados.

2.b.c) Tensorflow y Keras

La implementación del MLP con Tensorflow y Keras no es tan sencilla como la de Sklearn, principalmente porque Keras es mucho más flexible a la hora de crear modelos. Se ha intentado que la interfaz de la función sea compatible con la de Sklearn, y que ambas implementaciones sean más o menos equivalentes.

```

def train_keras_mlp(preprocess=preprocess, **kwargs):
    # Preprocess
    if preprocess:

```

```

        x = preprocess(x_train)
    else:
        x = x_train

    # Create model
    model = Sequential()
    model.add(Input(shape=(x.shape[-1],)))
    for size in kargs['hidden_layer_sizes']:
        model.add(Dense(size, kernel_initializer="he_normal", activation='relu'))
    model.add(Dense(10, kernel_initializer="he_normal", activation='softmax'))

    opt = Adam(learning_rate=kargs['learning_rate_init'])
    model.compile(
        loss='categorical_crossentropy',
        optimizer=opt,
        metrics=['accuracy']
    )

    # Train
    y_train_one_hot = to_categorical(y_train, num_classes=10)
    historial = model.fit(
        x,
        y_train_one_hot,
        epochs=kargs['max_iter'],
        batch_size=kargs['batch_size']
    )

    # Eval
    if preprocess:
        x = preprocess(x_test)
    else:
        x = x_test
    y_test_one_hot = to_categorical(y_test, num_classes=10)
    loss, accuracy = model.evaluate(x, y_test_one_hot, batch_size=10000)
    accuracy = tf.reduce_mean(accuracy)

    return model, historial, loss, accuracy

```

Para que la comparación sea justa, se ha utilizado la misma configuración que con Sklearn.

```

mlp_config = {
    'hidden_layer_sizes': [16],
    'learning_rate_init': 0.001,
    'max_iter': 300,
    'random_state': 25,
    'batch_size': 200,
    'tol': 0.0001,
}

```

A continuación se muestran los resultados obtenidos para los 4 modelos probados anteriormente. Como se puede ver, una ventaja que tiene Keras es que podemos obtener los resultados de evaluación del modelo en el *dataset* de *train* (y no solo el *loss*) a medida que se entrena el modelo. Esto nos permite ver de forma más clara cómo mejora el modelo con el tiempo.

Por ejemplo, se puede apreciar más claramente en la Figura 7 a la derecha que el modelo base con normalización aún podría estar bastantes más épocas entrenando hasta llegar al máximo posible de *accuracy*. También se puede apreciar muy claramente como el modelo con LDA llega al máximo posible de *accuracy* muy rápidamente, y ya no vuelve a mejorar más allá de este punto.

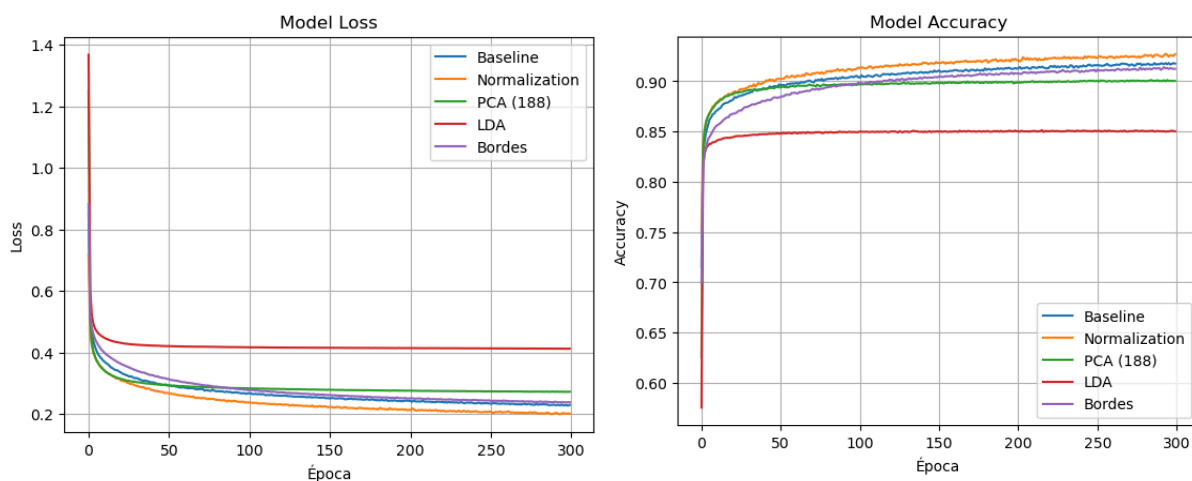


Figura 7: Resultados de *loss* y *accuracy* de entrenamiento para todos los modelos probados.

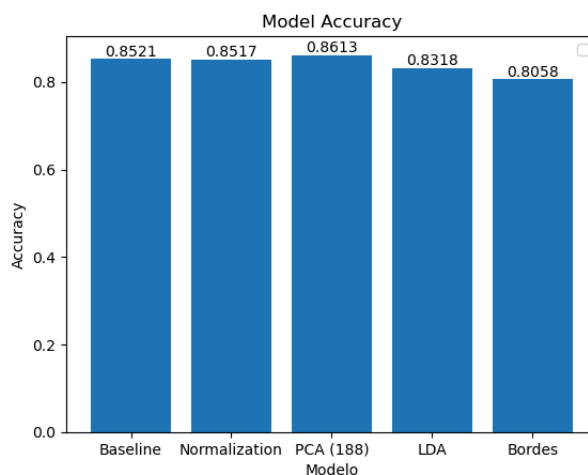


Figura 8: Resultados de *accuracy* de test de todos los modelos probados.

2.c) Red convolucional

A continuación se harán pruebas parecidas a las anteriores, pero utilizando redes convolucionales en vez de perceptrones multicapa. Estos tipos de redes permiten «compartir pesos» entre distintas posiciones de la imagen, reduciendo el tamaño efectivo del modelo sin repercutir en el rendimiento.

2.c.a) Baseline

Como antes, el código de entrenamiento y evaluación es muy personalizable para probar fácilmente distintos tipos de modelos.

```
def train_keras_cnn(preprocess=preprocess, **kwargs):
    # Preprocess
    if preprocess:
        x = preprocess(x_train)
    else:
```

```

x = x_train

# Create model
model = Sequential()
model.add(Input(shape=(x.shape[-2], x.shape[-1], 1)))

for conv in kargs['convolutions']:
    if 'dropout' in kargs:
        model.add(SpatialDropout2D(rate=kargs['dropout']))
    model.add(Conv2D(**conv))

if 'dropout' in kargs:
    model.add(SpatialDropout2D(rate=kargs['dropout']))

if kargs['final_pooling']:
    args = kargs['final_pooling']
    cls = args['type']
    del args['type']
    model.add(cls(**args))

model.add(Flatten()) # Flatten 2D to 1D
for size in kargs['hidden_layer_sizes']:
    model.add(Dense(size, kernel_initializer="he_normal", activation='relu'))
model.add(Dense(10, kernel_initializer="he_normal", activation='softmax')) #
Output

opt_cls = kargs['opt']

if isinstance(kargs['lr'], dict):
    args = kargs['lr']
    sch_cls = args['type']
    del args['type']
    lr = sch_cls(**args)
else:
    lr = kargs['lr']

opt = opt_cls(learning_rate=lr)

if isinstance(kargs['loss'], dict):
    args = kargs['loss']
    loss_cls = args['type']
    del args['type']
    loss = loss_cls(**args)
else:
    loss = kargs['loss']

model.compile(loss=loss, optimizer=opt, metrics=['accuracy'])

# Train
y_train_one_hot = to_categorical(y_train, num_classes=10)
historial = model.fit(
    x, y_train_one_hot,
    epochs=kargs.get('epochs', 10),
    batch_size=kargs.get('batch_size', 128)
)

```

```

# Eval
if preprocess:
    x = preprocess(x_test)
else:
    x = x_test
y_test_one_hot = to_categorical(y_test, num_classes=10)
loss, accuracy = model.evaluate(x, y_test_one_hot, batch_size=60000)
accuracy = tf.reduce_mean(accuracy)

return model, historial ,loss, tf.reduce_mean(accuracy)

```

La configuración a probar es la que se indica en la práctica. Una única convolución de 3×3 con 16 filtros, un *max pool* de 2×2 , y una capa oculta de 20 neuronas.

```

mlp_config = {
    'convolutions': [
        {
            'filters': 16,
            'kernel_size': (3,3)
        }
    ],
    'final_pooling': {
        'type': MaxPooling2D,
        'pool_size': (2, 2)
    },
    'hidden_layer_sizes': [20],
    'opt': SGD,
    'lr': 0.1,
    'loss': 'categorical_crossentropy',
    'epochs': 10,
    'batch_size': 128
}

cnn, cnn_hist, cnn_loss, cnn_score = train_keras_cnn(
    functools.partial(preprocess_norm, collapse=False),
    **mlp_config
)

```

Se puede comprobar en las gráficas siguientes que el modelo aprende bastante, aunque está claro que 10 épocas es una cantidad muy pequeña para el entrenamiento, y que el modelo podría aprender más. El modelo acaba teniendo un *accuracy* de test del 89,30%.

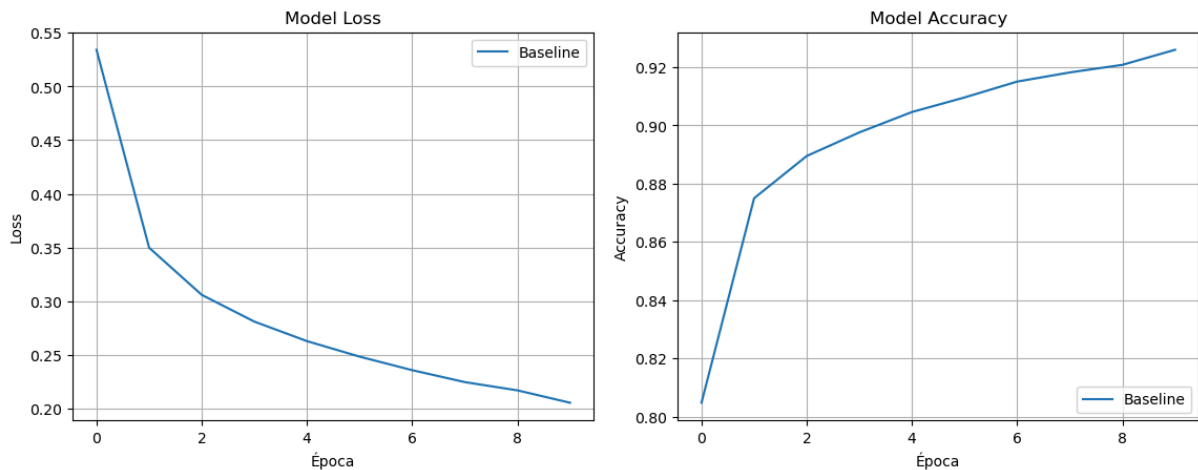


Figura 9: Resultados de *loss* y *accuracy* de entrenamiento para el modelo básico.

2.c.b) Mejoras

- **Entrenar durante mucho más tiempo:** es obvio que el *baseline* está poco entrenado, por lo que la primera mejora obvia a probar es dejarlo entrenar mucho más tiempo. En específico, como se puede ver en la siguiente gráfica, tras 100 épocas el modelo alcanza un *accuracy* del 100% en el entrenamiento, y un *accuracy* del 89,08% en el test. Como se puede ver, hay una cantidad significativa de *overfitting*.

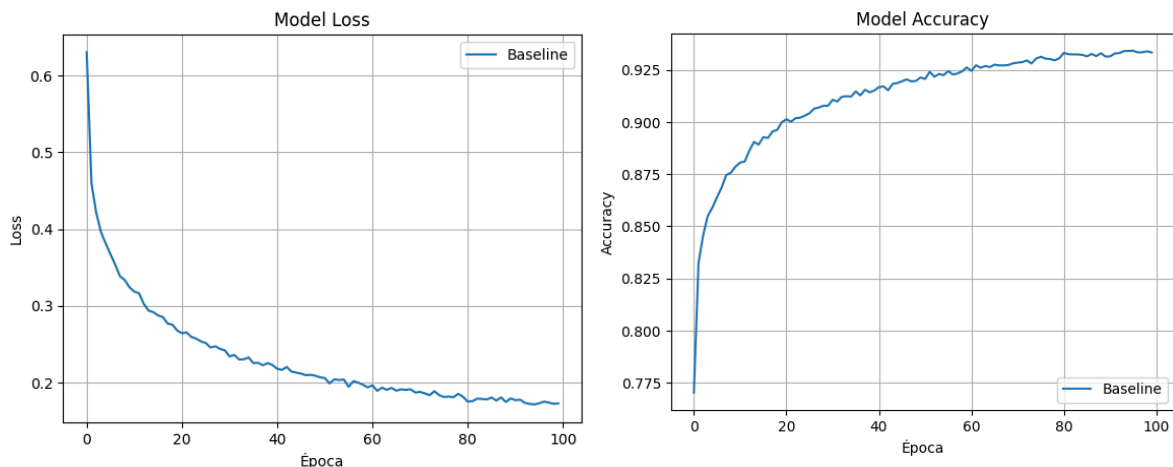


Figura 10: Resultados de *loss* y *accuracy* de entrenamiento para el modelo básico.

- **Ajustar el optimizador, *learning rate*, y regularización (*dropout* y *L2*):** básicamente consiste en utilizar el optimizador Adam en vez de SGD, y ajustando el *learning rate* de forma correspondiente (más específicamente, lo mantenemos a 0.01 las primeras 40 épocas, a 0.001 las siguientes 40 épocas, y a 0.0001 las últimas 20 épocas). Teniendo en cuenta el *overfitting* que se va a producir, se añade *dropout*, y regularización L2. Todo esto consigue que suba el *accuracy* del test a 90,35% sin añadir ningún parámetro entrenable.

```
mlp_config = {
    'convolutions': [
        {
            'filters': 16,
            'kernel_size': (3,3),
            'kernel_regularizer': L2(0.05)
        }
    ]
}
```

```

    }
],
'dropout': 0.1,
'final_pooling': {
    'type': MaxPooling2D,
    'pool_size': (2, 2)
},
'hidden_layer_sizes': [20],
'opt': Adam,
'lr': {
    'type': PiecewiseConstantDecay,
    'boundaries': [469*40, 469*80],
    'values': [0.01, 0.001, 0.0001],
},
'loss': 'categorical_crossentropy',
'epochs': 100,
'batch_size': 128
}

```

Nótese en las gráficas de *loss* y *accuracy* como el modelo mejora repentinamente en la época 40 cuando se reduce repentinamente el *learning rate*.

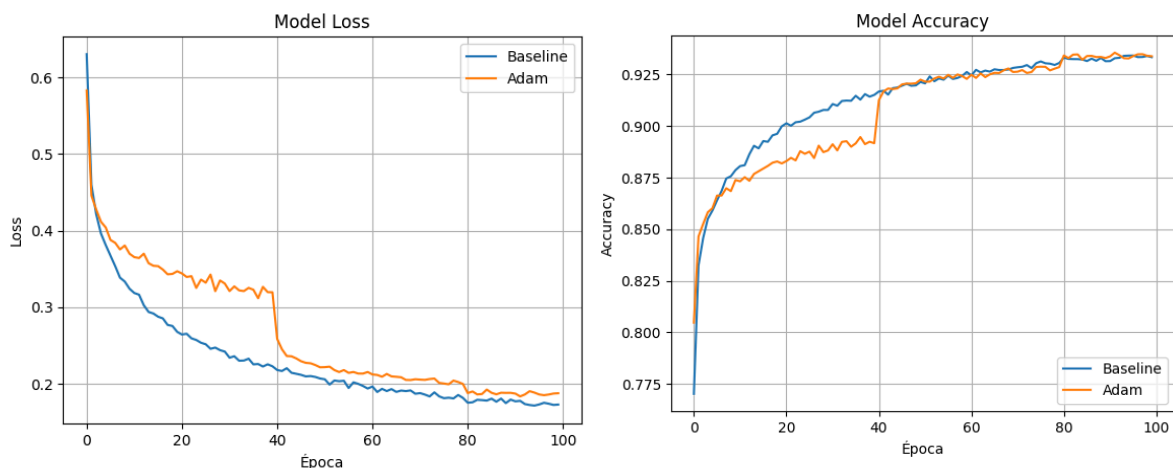


Figura 11: Resultados de *loss* y *accuracy* de entrenamiento para el modelo con optimizador Adam y regularización elevada.

- **Aumentar el número de convoluciones:** simplemente aumentar el número de convoluciones y el número de filtro por convolución permite mejorar los resultados en el test hasta el 91,74%. En este caso se muestran 4 convoluciones de 32 filtros de tamaño 4×4 . Como son varias convoluciones, para que convergan más fácilmente se utiliza *batch normalization* después de cada convolución. Además, como son varias capas, se utiliza la función de activación ReLU en cada convolución para así añadir no linealidad al modelo.

```

mlp_config = {
    'convolutions': [
        {
            'filters': 32,
            'kernel_size': (4,4),
            'activation': 'relu',
            'kernel_regularizer': L2(0.01),

```

```

        'use_bias': False
    },
    {
        'filters': 32,
        'kernel_size': (4,4),
        'activation': 'relu',
        'kernel_regularizer': L2(0.01),
        'use_bias': False
    },
    {
        'filters': 32,
        'kernel_size': (4,4),
        'activation': 'relu',
        'kernel_regularizer': L2(0.01),
        'use_bias': False
    },
    {
        'filters': 32,
        'kernel_size': (4,4),
        'activation': 'relu',
        'kernel_regularizer': L2(0.01),
        'use_bias': False
    }
],
'dropout': 0.08,
'batchnorm': True,
'final_pooling': {
    'type': MaxPooling2D,
    'pool_size': (2, 2)
},
'hidden_layer_sizes': [128],
'opt': SGD,
'lr': {
    'type': PiecewiseConstantDecay,
    'boundaries': [469*40, 469*80],
    'values': [0.1, 0.01, 0.001],
},
'loss': 'categorical_crossentropy',
'epochs': 100,
'batch_size': 128
}

```

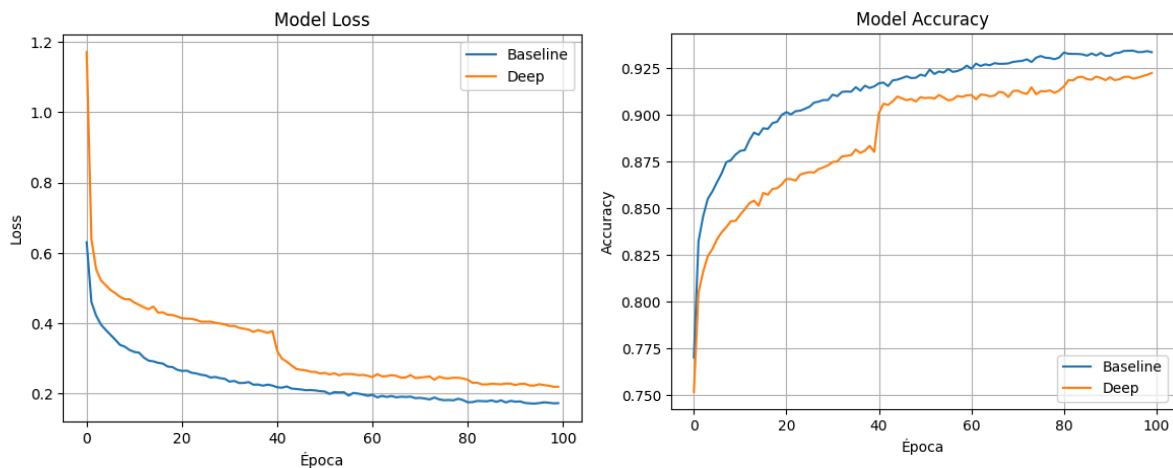


Figura 12: Resultados de *loss* y *accuracy* de entrenamiento para el modelo con optimizador Adam y regularización elevada.

Se muestran a continuación los resultados de *accuracy* en el *dataset* de test en los 3 modelos probados. Como se puede ver, el que consiguió mejores resultados es el modelo grande con varias convoluciones. Nótese que habríamos probado más modelos, pero Tensorflow no detectaba la GPU de nuestros portátiles, y en Collab nos ponían límites de uso de GPU.

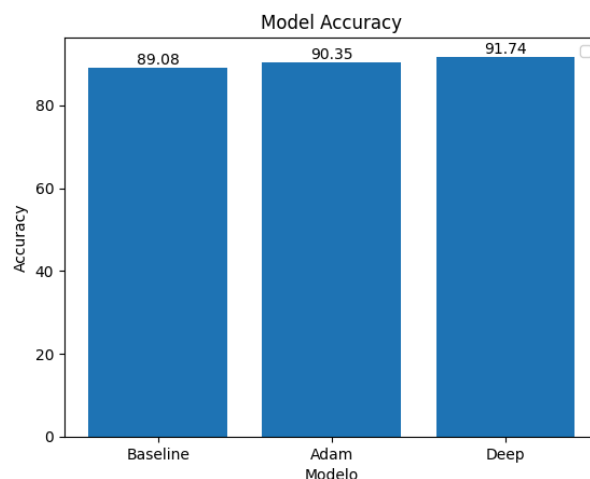


Figura 13: Resultados de *accuracy* de test de todos los modelos probados.

2.d) Conclusión

En conclusión, las redes convolucionales se ha visto empíricamente que son una forma muy efectiva de tratar con problemas de visión por computador, permitiendo conseguir resultados decentes muy rápidamente, y conseguir muy buenos resultados con un entrenamiento más largo.

3) NLP

Vamos a realizar el entrenamiento de una red neuronal profunda para que sea un clasificador binario de sentimiento en base a reviews de productos.

Las redes neuronales más populares a día de hoy son redes recursivas (los transformadores por ejemplo tienen bloques en los que aplican sus mecanismos como la atención para sus decisiones). En este caso vamos a emplear redes recursivas simples, como son las RNN, las LSTM y las GRU.

Para facilitar este apartado, tenemos ya dos *datasets* separados en train y test de reviews. En estos *datasets* tenemos las labels, el título y las reviews y sabemos que las clases están balanceadas

Para el procedimiento de este apartado vamos a realizar la creación de un modelo con unas características concretas que vamos a usar como Baseline y a partir de ahí vamos a intentar mejorar los resultados siguiendo unas técnicas en concreto

3.a) Evaluación

Como Keras, al ser un entorno más completo, ejecuta el training y el testing con batches y realiza casi todo el proceso de evaluación por sí solo, nosotros en este caso vamos a usar para evaluar como criterio definitivo el **porcentaje de accuracy que se obtiene en el test**.

3.b) Preprocesamiento

Para realizar el preprocesamiento realmente esta vez no tenemos que hacer mucho. En este tipo de problemas lo que habría que hacer sería lo siguiente:

- Eliminar posibles valores nulos.
- Reducir el vocabulario con técnicas estándar como sería el quitar las mayúsculas.

En este caso quitamos los nulos del *dataset* con un **.dropna()**. Sin embargo, como veremos en un futuro, una de las capas de nuestro modelo básico ya realiza el resto del preprocesamiento por nosotros, por lo que la siguiente parte de reducir el vocabulario no hace falta hacerla

3.c) Resultados

A lo largo del informe, veremos que se usan unas gráficas en concreto, para visualizar tanto como baja el loss, cómo como sube el accuracy y unos barplots para ver el accuracy del test, el código para dichas gráficas es el siguiente:

```
#historial = model1.fit()

def plot_accuracy(historiales, nombres, path):
    """
    Hace un plot de la evolución de los accuracies según los históricos del fit
    """
    for i, historial in enumerate(historiales):
        plt.plot(historial.history['accuracy'], label = nombres[i])
    #plt.plot(history.history['val_accuracy'])
    plt.title('model accuracy')
    plt.ylabel('accuracy')
    plt.xlabel('epoch')
    if len(historiales) < 13:
        plt.legend(loc='upper left')
    plt.savefig(f"Accuracy_{path}.png", bbox_inches='tight')
    plt.show()

def plot_loss(historiales, nombres, path):
    """
    Hace un plot de la evolución de los losses según los históricos del fit
    """
    for i, historial in enumerate(historiales):
        plt.plot(historial.history['loss'], label = nombres[i])
    #plt.plot(history.history['val_accuracy'])
    plt.title('model loss')
```

```

plt.ylabel('loss')
plt.xlabel('epoch')
if len(historiales) < 13:
    plt.legend(loc='upper right')
plt.savefig(f"Loss_{path}.png", bbox_inches='tight')
plt.show()

def plot_test_accuracy(accuracies, nombres, path):
    """
    Hace un plot de los accuracies de los tests
    """
    plt.bar(x=nombres, height=accuracies, )
    #plt.plot(history.history['val_accuracy'])
    plt.title('model accuracy')
    plt.ylabel('accuracy')
    plt.xlabel('epoch')
    #plt.legend(loc='upper left')
    plt.savefig(f"Test_acc_{path}.png", bbox_inches='tight')
    plt.show()

```

3.c.a) Baseline

Para crear un baseline a partir del que trabajar vamos a diseñar un modelo particular. En el pdf se nos describe el modelo con las siguientes características:

- Capa de tokenización: Es la capa que se encarga de transformar los textos a «tokens», que son subdivisiones que se usan para luego codificar a un lenguaje que la IA pueda entender. Para esta capa usaremos:
 - keras.TextVectorization como clase
 - 1000 como tamaño máximo de vocabulario (max tokens)
 - 50 como tamaño máximo de secuencia (ouput sequence length)
- Capa de Embedding: En la capa de embedding es donde se traducen los tokens a sus vectores de embeddings. Estos demuestran en un lenguaje que la máquina entiende (números) lo que queremos comunicar
 - keras.Embedding como clase
 - output_dim a tamaño 16 (capa de salida)
- Capa recursiva: En esta capa es donde se realizará el aprendizaje. Las capas recursivas se caracterizan por tener como inputs los outputs de una ejecución anterior.
 - keras.SimpleRNN como clase
 - units a tamaño 16 (numero de capas recurrentes)
- Collapse (extra): Esta capa es en la que transformamos los valores que teníamos antes (vectores de números) en nuestra clasificación binaria. Para ello vamos a usar una capa dense que colapsa las 16 dimensiones que teníamos antes en 1 usando una función sigmoide para separar en 0 y en 1.
 - keras.Dense como capa de activación
- Optimizador: el optimizador es la entidad encargada de supervisar el entrenamiento y dictaminar como se realiza. En este caso tenemos que usar:
 - Adam como estrategia de entrenamiento
 - 128 de batch size
 - 10 épocas de entrenamiento

3.c.a.a) Código

La traducción de lo dicho anteriormente a código es la siguiente:

```
class modeloBasico:
    """
    Modelo básico con los parametros que se indican en el pdf

    - max_tokens: tamaño de vocabulario
    - output_sequence_length: tamaño maximo del texto de entrada
    - output_dim: dimensiones de salida de la capa de embeddings
    - units: numero de unidades de la capa recursiva
    - vocab: vocabulario al que adaptar la capa de vectorizacion de texto
    """
    def __init__(self,
                  max_tokens: int = 1000,
                  output_sequence_length: int = 50,
                  output_dim: int = 16,
                  units: int = 16,
                  vocab = None
                  ) -> None:
        self._tokenizer_l = layers.TextVectorization(
            max_tokens=max_tokens,
            output_sequence_length= output_sequence_length,
        )
        self._tokenizer_l.adapt(vocab)
        self._embedding_l = layers.Embedding(
            input_dim=max_tokens,
            output_dim=output_dim
        )
        self._rnn_l = layers.SimpleRNN(
            units=units,
        )
        self._dense_l = layers.Dense(1, activation="sigmoid")
        self.model = keras.Sequential(
            [self._tokenizer_l, self._embedding_l, self._rnn_l, self._dense_l]
        )
        self.model.compile(
            optimizer = "adam", loss="binary_crossentropy", metrics=["accuracy",
keras.metrics.F1Score]
        )

    def summary(self):
        """
        Devuelve el summary de las capas del modelo
        """
        return self.model.summary()

    def fit(self, X, Y, epochs = 10, batch_size = 128):
        """
        Hace el fitting del modelo en base a X e Y. Devuelve el histórico del
        entrenamiento
        """
        return self.model.fit(
            X, Y,
            epochs=epochs,
```

```

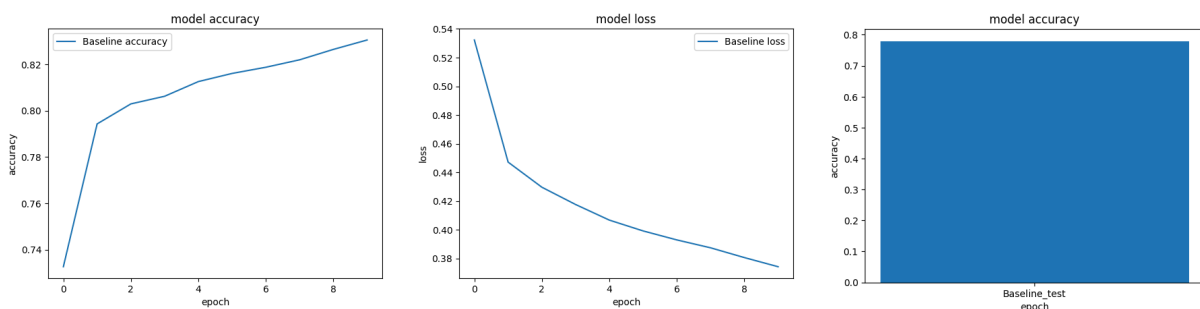
        batch_size=batch_size,
    )

    def test(self, X_test, Y_test):
        """
        Devuelve el accuracy del test
        """
        #devuelve [loss, acc, extras] entonces nos quedamos con el accuracy del
        test :D
        return self.model.evaluate(X_test, Y_test)[1]

```

3.c.a.b) Resultados

Los resultados de ejecutar el baseline son los siguientes:



Baseline_test = 0.7796249985694885

Podemos observar que con los parametros por defecto, nos aproximamos a un los de 0.37 y a una accuracy en el training de 0.83. Sin embargo, nuestra métrica oficial baseline es de **0.78 de accuracy en el test**

3.c.b) Mejoras

Para realizar las mejoras vamos a realizar ciertos cambios siguiendo las recomendaciones del pdf. Para ello dividiremos la exploración de mejoras de la siguiente manera

- Configuración del tokenizador
- Capas recurrentes, tipos y tamaños
- Otras capas y Dropout
- Otros optimizadores
- Otras mejoras

Se probaran las cosas por separado porque realizar combinaciones de todo puede resultar en tiempos de computación desorbitados.

De nuevo, el criterio de evaluación es la accuracy que se deduce del test. Recaltar que en todas las mejoras se realizarán comparativas con el baseline.

Para realizar este tipo de mejora se realiza una clase genérica en la que se pueden introducir simplemente los diccionarios con los cambios como argumentos para poder simplificar los cálculos

```

class modeloConMejorasBasico:
    """
    Modelo diseñado para variar los parametros. Por defecto los valores son los del
    pdf

    - max_tokens: tamaño de vocabulario
    - output_sequence_length: tamaño maximo del texto de entrada
    - padding: indica si hace que el tamaño de salida de la capa de vectorización
      sea siempre el máximo
    - preprocessing: indica el tipo de preprocesamiento hacer en la capa de
    vectorizacion
    - output_dim: dimensiones de salida de la capa de embeddings
    - units: numero de unidades de la capa recursiva
    - vocab: vocabulario al que adaptar la capa de vectorizacion de texto
    - Recursive_LayerClass: clase que se usa para la recursión del modelo
    - Dropout: indica el dropout de las capas recursivas (por defecto 0)
    - ExtraLayers: array de capas extra a meter antes de la capa que colapsa
    para dos clases
    """
    def __init__(self,
                  max_tokens = 1000,
                  output_sequence_length = 50,
                  padding = False,
                  preprocessing = "lower_and_strip_punctuation",
                  output_dim = 16,
                  units = 16,
                  vocab = None,
                  Recursive_LayerClass = layers.SimpleRNN,
                  dropout = False,
                  ExtraLayers = [],
                  ) -> None:
        self._tokenizer_l = layers.TextVectorization(
            max_tokens=max_tokens,
            output_sequence_length=output_sequence_length,
            standardize = preprocessing,
            pad_to_max_tokens = padding
        )
        self._tokenizer_l.adapt(vocab)

        self._embedding_l = layers.Embedding(
            input_dim=max_tokens,
            output_dim=output_dim
        )

        self._recursive_layer = Recursive_LayerClass(
            units=units,
            recurrent_dropout = dropout if dropout is not None else 0,
            dropout = dropout if dropout is not None else 0,
        )

        self._collapse_layer = layers.Dense(1, activation="sigmoid")

        self.model: keras.Sequential = keras.Sequential(
            [self._tokenizer_l, self._embedding_l, self._recursive_layer, *ExtraLayers,
            self._collapse_layer]

```

```

    )

    self.model.compile(
        optimizer = "adam", loss="binary_crossentropy", metrics=["accuracy",
keras.metrics.F1Score]
    )

    def summary(self):
        """
        Devuelve el summary de las capas del modelo
        """
        return self.model.summary()

    def fit(self, X, Y, epochs = 10, batch_size = 128):
        """
        Hace el fitting del modelo en base a X e Y. Devuelve el histórico del
entrenamiento
        """
        return self.model.fit(
            X, Y,
            epochs=epochs,
            batch_size=batch_size,
        )

    def test(self, X_test, Y_test):
        """
        Devuelve el accuracy del test
        """
        #devuelve [loss, acc, extras] entonces nos quedamos con el accuracy del
test :D
        return self.model.evaluate(X_test, Y_test)[1]

```

A mayores, definimos una función genérica para hacer todas las pruebas automáticamente

```

def test_all(params):
    """
    Funcion que crea un modelo con los paramentros, lo entrena, evalua el test y
guarda los
datos a partir de un array de parametros
    """

    results = {
        "history": [Baseline_train],
        "test_acc": [Baseline_test],
        "params": ["Por defecto"],
        "names": ["Baseline"],
    }
    for i, param_comb in enumerate(params):

        model = modeloConMejorasBasico(**param_comb, vocab=X)

        results["history"].append(model.fit(X, Y))

        results["test_acc"].append(model.test(X_test, Y_test))

```

```

results["params"].append(param_comb)

results["names"].append(f"pl_c_{i}")

results["best_test_acc_index"] = np.argmax(results["test_acc"])
results["best_test_acc"] = results["test_acc"][results["best_test_acc_index"]]
results["best_params"] = results["params"][results["best_test_acc_index"]]

return results

```

A mayores, implementamos código auxiliar para generar los diccionarios de combinaciones:

```

import itertools

def generate_param_combinations(dic):
    """
    Convierte un diccionario de arrays de parametros en un array de diccionarios de
    combinaciones
    """

    param_names = list(dic.keys())
    param_values = list(dic.values())
    combinations = list(itertools.product(*param_values))

    result = []
    for combo in combinations:
        param_dict = dict(zip(param_names, combo))
        result.append(param_dict)

    return result

```

3.c.b.a) Configuración del tokenizador

La capa de tokenización es la que dictamina como transformamos los textos escritos en humano a tokens. En este proceso se realizan muchos procesos intrínsecos con los que podemos probar a mejorar.

- Vamos a probar a variar el tamaño de vocabulario para ver si que la IA tenga más o menos palabras influye. Vamos a probar 256, 512, 1000 y 1500
- Vamos a probar a variar el tamaño máximo de secuencia. Vamos a probar tamaños 25, 50, 75 y 100
- Como ya se comentó anteriormente, en esta capa de manera intrínseca se realiza un preprocesamiento. Por defecto se quitan los signos de puntuación y las mayúsculas para quitar posibles elementos del vocabulario que distraigan. En este caso vamos a probar que es mejor, quitar ambos, quitar puntuación, quitar solo mayúsculas o no quitar ninguno.
- Por último, sabemos que estamos metiendo cadenas de tamaño variable. Esto produce vectores de tamaño variable. Por ende vamos a probar a meterle padding (o no) que fuerza a que todos los vectores sean de tamaño máximo para ver si así simplificamos las cosas para nuestro aprendizaje

3.c.b.a.a) Código

Vamos a probar las siguientes combinaciones:

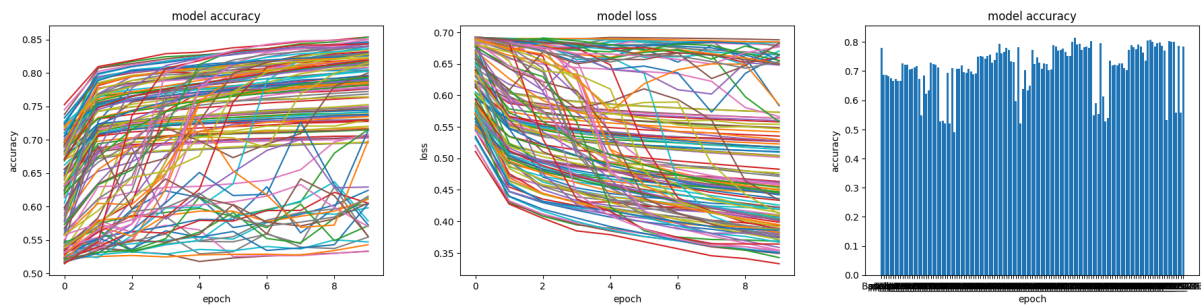
```

params_p1 = {
    "max_tokens": (256, 512, 1000, 1500),
    "output_sequence_length": (25, 50, 75, 100),
    "preprocessing": ("lower_and_strip_punctuation", "lower", "strip_punctuation",
None),
    "padding": (False, True),
}
Comb_params_p1 = generate_param_combinations(params_p1)

```

3.c.b.a.b) Resultados

En este apartado es en el que más combinaciones de parametros se ejecutan



```

Best_param_index = 82,
Best_test_accuracy = 0.8152499794960022,
Best_params = {'max_tokens': 1000,
'output_sequence_length': 75,
'preprocessing': 'lower_and_strip_punctuation',
'padding': True})

```

Podemos observar las siguientes conclusiones:

- Los modelos que tenían muy poco output_sequence_length, muy poca estandarizacion o muy poco tamaño de vocabulario son los que peor actuan
- Podemos ver que a nivel general hay varios modelos que alcanzan accuracies similares al baseline
- El **mejor modelo** de la prueba 1 alcanza un accuracy en el test de **0.815** lo que supone un incremento de un **3.5% frente al baseline**. Es un incremento suficiente como para no poder ignorarlo.

3.c.b.b) Capas recurrentes, tipos y tamaños

Como hemos dicho antes, en el diagrama de sistema que teníamos como baseline, los encargados de realizar el aprendizaje son las capas recursivas. Como usabamos RNN y 16 unidades como base vamos a probar las siguientes variaciones:

- Vamos a probar a variar el tipo de capa recursiva (RNN, LSTM o GRU)
 - RNN es la version «simple» de una capa recursiva
 - LSTM y GRU son mejoras sobre una RNN que incorporan diferentes tipos de gates para de alguna manera tratar con la «memoria» y/o los «updates»
- Vamos a probar a variar la cantidad de unidades que hay en cada capa (16, 32, 64, 128)

3.c.b.b.a) Código

En este caso las combinaciones de paramentros son estas

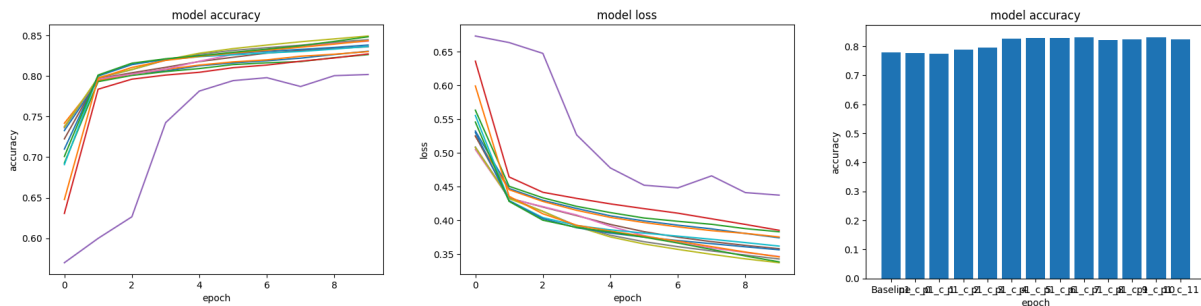
```

params_p2 = {
    "Recursive_LayerClass": (layers.SimpleRNN, layers.LSTM, layers.GRU),
    "units": (16, 32, 64, 128),
}
Comb_params_p2 = generate_param_combinations(params_p2)

```

3.c.b.b) Resultados

En este apartado es en el que más combinaciones de parametros se ejecutan



```

Best_param_index = 11,
Best_test_acc = 0.8317499756813049,
Best_param_combination = {'Recursive_LayerClass': keras.src.layers.rnn.gru.GRU,
                           'units': 64}

```

A diferencia del apartado anterior, donde no se podía observar una tendencia tan clara, en este caso podemos observar una mejora en casi cualquier combinación de parametros respecto al baseline

- Se observa una tendencia general en la que la accuracy aumenta y «converge» de manera más rápida al incrementar la complejidad y el tamaño de las capas
- De manera analoga, se puede observar lo mismo con la bajada del loss
- Los incrementos de complejidad y de tamaño acaban convergiendo, es decir, no se observa la tendencia de que aumentar descomunalmente el tamaño merezca la pena porque produce mejoras cada vez menores
- El **mejor modelo** de la prueba 2 alcanza un accuracy en el test de **0.832** lo que supone un incremento de un **5% frente al baseline**. Es un incremento suficiente como para no poder ignorarlo. Es un incremento más notable que con la prueba anterior

3.c.b.c) Otras capas y Dropout

La siguiente posible mejora es la posibilidad de complicar el sistema de la IA añadiéndole complejidad de capas. Para ello vamos a dar la posibilidad de añadir toda la cantidad de capas posibles. Como añadir capas constituye un aumento de complejidad computacional, se limitan los ejemplos a probar a aquellos que de manera empírica proporcionaron resultados correctos.

- Se añaden capas extra Dense y/o Dropout

A mayores, dentro de las capas recursivas podemos meter dropout, de tal manera que se «olviden» conexiones durante el entrenamiento para así obtener resultados más generales.

- Se prueban a meter 0.1, 0.2 y 0.3 de dropout en las capas recursivas

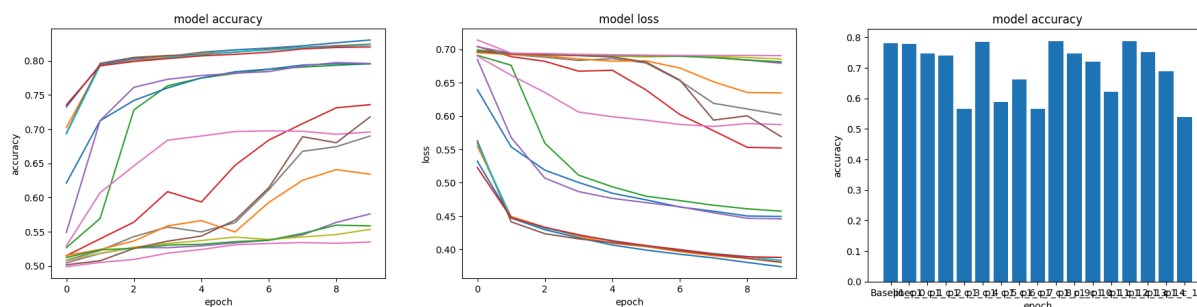
3.c.b.c.a) Código

Antes de realizar la prueba general, se ha experimentado con meter más o menos capas, sin embargo no se observaban mejoras aparentes, por lo que se van a realizar pruebas solo con las más prometedoras

```
arrays_layers = [  
    [ layers.Dense(16, activation="relu"), layers.Dropout(0.1) ],  
    [ layers.Dense(16, activation="relu")],  
    [ layers.Dense(32, activation="relu"), layers.Dropout(0.1) ],  
    [ layers.Dense(32, activation="relu"), layers.Dropout(0.1), layers.Dense(16,  
activation="relu")],  
]  
params_p3 = {  
    "ExtraLayers": arrays_layers,  
    "dropout": (False, 0.1, 0.2, 0.3)  
}  
Comb_params_p3 = generate_param_combinations(params_p3)
```

3.c.b.c.b) Resultados

En este apartado es en el que más combinaciones de parametros se ejecutan



```
Best_param_index = 13,  
Best_test_acc = 0.7877500057220459,  
Best_param_combination = {'ExtraLayers': [<Dense name=dense_144, built=True>,  
    <Dropout name=dropout_2, built=True>,  
    <Dense name=dense_145, built=True>],  
    'dropout': False}
```

A nivel de resultados podemos ver los siguiente:

- El meter las capas y el dropout, sobre todo con un tamaño de units de 16 por defecto, genera resultados raros
- La mejor combinación nos ha dado un **0.788** de precision en test, apenas un **0.5%** mejor que la baseline, por lo que no podemos decir que sea extremadamente relevante
- El dropout en papel funciona para generalizar, en este caso al ser los units tan pequeños no parece ayudar, pro con pruebas para tamaños más grandes el dropout si que ayuda a generalizar

3.c.b.d) Otros optimizadores

En el baseline se nos pide que usemos Adam como optimizador. En la realidad, el optimizador más utilizado es el AdamW porque es el que generalmente mejor funciona. Sin embargo este optimizador es más complejo.

A mayores, la estrategia de reducir el learningrate se utiliza para sacar al modelo cuando se «estanca» en el aprendizaje. En este caso hacemos que cada 3 épocas, el learning rate baje en una escala de 10.

3.c.b.d.a) Código

Para probar el optimizador y los learning rates modificamos el código:

```
class modeloBasicoMasOptimizador:
    """
    Modelo básico con los parametros que se indican en el pdf pero con otro optimizador

    - max_tokens: tamaño de vocabulario
    - output_sequence_length: tamaño maximo del texto de entrada
    - output_dim: dimensiones de salida de la capa de embeddings
    - units: numero de unidades de la capa recursiva
    - vocab: vocabulario al que adaptar la capa de vectorizacion de texto
    """
    def __init__(self,
                  max_tokens = 1000,
                  output_sequence_length = 50,
                  output_dim = 16,
                  units = 16,
                  vocab = None
                  ) -> None:
        self._tokenizer_l = layers.TextVectorization(
            max_tokens=max_tokens,
            output_sequence_length= output_sequence_length,
        )
        self._tokenizer_l.adapt(vocab)
        self._embedding_l = layers.Embedding(
            input_dim=max_tokens,
            output_dim=output_dim
        )
        self._rnn_l = layers.SimpleRNN(
            units=units,
        )
        self._dense_l = layers.Dense(1, activation="sigmoid")
        self.model = keras.Sequential(
            [self._tokenizer_l, self._embedding_l, self._rnn_l, self._dense_l]
        )

        """
        Usamos el optimizador AdamW, a mayores, hacemos que en las iteraciones en
        las que empíricamente se nos queda atascado el modelo se le baje el learning rate
        para que siga aprendiendo
        """

        lr = PiecewiseConstantDecay([563*3, 563*7], [0.01, 0.001, 0.0001])
        opt = AdamW(learning_rate=lr)
        self.model.compile(
            optimizer = opt,
            loss="binary_crossentropy",
            metrics=["accuracy", keras.metrics.F1Score]
        )
```

```

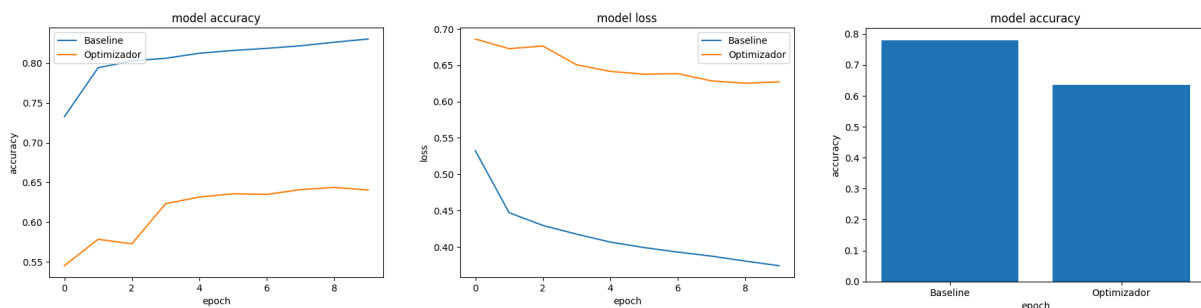
def summary(self):
    """
    Devuelve el summary de las capas del modelo
    """
    return self.model.summary()

def fit(self, X, Y, epochs = 10, batch_size = 128):
    """
    Hace el fitting del modelo en base a X e Y. Devuelve el histórico del
    entrenamiento
    """
    return self.model.fit(
        X, Y,
        epochs=epochs,
        batch_size=batch_size,
    )

def test(self, X_test, Y_test):
    """
    Devuelve el accuracy del test
    """
    #devuelve [loss, acc, extras] entonces nos quedamos con el accuracy del
    test :D
    return self.model.evaluate(X_test, Y_test)[1]

```

3.c.b.d.b) Resultados



```
resultados_test = 0.6340000033378601
```

- Podemos observar que para nuestro caso, al menos con la versión original, el empleo de AdamW no es óptimo. Es posible que necesite más épocas para salir del calentamiento o que se ajuste alguno de sus parámetros
- El ajuste de learning rate se ve eclipsado por el AdamW, sin embargo de pruebas anteriores y los resultados del apartado de imágenes, podemos decir que no es mala idea implementarlo

3.c.c) Mejor Modelo

Para realizar el mejor modelo vamos a combinar los mejores parámetros obtenidos hasta el momento:

- “max_tokens”: 1000,
- “output_sequence_length”: 75,
- “preprocessing”: “lower_and_strip_punctuation”,

- “padding”: True,
- “Recursive_LayerClass”: layers.GRU,
- “units”: 64
- “Learning rate” adaptado para disminuirse cuando se estanque

3.c.c.a) Código

El código de la clase

```
class modeloConMejorasBasicoYLearningRate:
    """
    Modelo diseñado para variar los parametros. Por defecto los valores son los del
    pdf. Tiene implementado learning rate decay

    - max_tokens: tamaño de vocabulario
    - output_sequence_length: tamaño maximo del texto de entrada
    - padding: indica si hace que el tamaño de salida de la capa de vectorización
    sea siempre el máximo
    - preprocessing: indica el tipo de preprocesamiento hacer en la capa de
    vectorizacion
    - output_dim: dimensiones de salida de la capa de embeddings
    - units: numero de unidades de la capa recursiva
    - vocab: vocabulario al que adaptar la capa de vectorizacion de texto
    - Recursive_LayerClass: clase que se usa para la recursión del modelo
    - Dropout: indica el dropout de las capas recursivas (por defecto 0)
    - ExtraLayers: array de capas extra a meter antes de la capa que colapsa
    para dos clases
    """
    def __init__(self,
                  max_tokens = 1000,
                  output_sequence_length = 50,
                  padding = False,
                  preprocessing = "lower_and_strip_punctuation",
                  output_dim = 16,
                  units = 16,
                  vocab = None,
                  Recursive_LayerClass = layers.SimpleRNN,
                  dropout = False,
                  ExtraLayers = [],
                  ) -> None:
        self._tokenizer_l = layers.TextVectorization(
            max_tokens=max_tokens,
            output_sequence_length= output_sequence_length,
            standardize = preprocessing,
            pad_to_max_tokens = padding
        )
        self._tokenizer_l.adapt(vocab)

        self._embedding_l = layers.Embedding(
            input_dim=max_tokens,
            output_dim=output_dim
        )

        self._recursive_layer = Recursive_LayerClass(
            units=units,
```

```

        recurrent_dropout = dropout if dropout is not None else 0,
        dropout = dropout if dropout is not None else 0,
    )

    self._collapse_layer = layers.Dense(1, activation="sigmoid")

    self.model: keras.Sequential = keras.Sequential(
        [self._tokenizer_l, self._embedding_l, self._recursive_layer, *ExtraLayers,
self._collapse_layer]
    )

    lr = PiecewiseConstantDecay([563*3, 563*7], [0.01, 0.001, 0.0001])
    opt = Adam(learning_rate=lr)
    self.model.compile(
        optimizer = opt,
        loss="binary_crossentropy",
        metrics=["accuracy", keras.metrics.F1Score]
    )

    def summary(self):
        """
        Devuelve el summary de las capas del modelo
        """
        return self.model.summary()

    def fit(self, X, Y, epochs = 10, batch_size = 128):
        """
        Hace el fitting del modelo en base a X e Y. Devuelve el histórico del
        entrenamiento
        """
        return self.model.fit(
            X, Y,
            epochs=epochs,
            batch_size=batch_size,
        )

    def test(self, X_test, Y_test):
        """
        Devuelve el accuracy del test
        """
        #devuelve [loss, acc, extras] entonces nos quedamos con el accuracy del
        test :D
        return self.model.evaluate(X_test, Y_test)[1]

```

El código de los parámetros

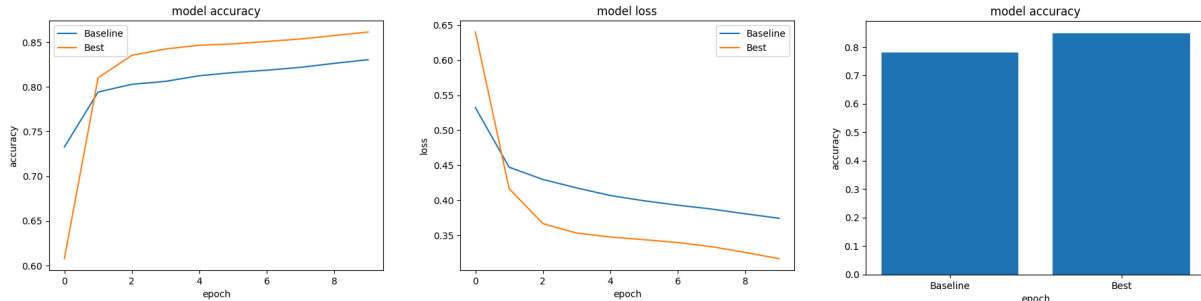
```

best_params = {
    'max_tokens': 1000,
    'output_sequence_length': 75,
    'preprocessing': 'lower_and_strip_punctuation',
    'padding': True,
    'Recursive_LayerClass': layers.GRU,
    'units': 64
}

```

```
best_model = modeloConMejorasBasico(**best_params, vocab=X)
best_train_res = best_model.fit(X, Y)
best_test_res = best_model.test(X_test, Y_test)
```

3.c.c.b) Resultados



```
best_test_res = 0.8483750224113464
```

Combinando todos los mejores parámetros conseguimos **el mejor modelo**. Este presenta un test_accuracy del **0.848**. Comparado con el **0.78** del baseline supone casi un **7% de mejora frente al baseline**

3.c.d) Otras mejoras

Hemos estado empleando versiones simplificadas de los sistemas actuales. Estos sistemas (los transformadores) son bastante más complejos de implementar. Sin embargo, para este apartado vamos a emplear varias de las técnicas que comprenden a un transformador para ver como resultaría el resultado. De esta manera creamos un pseudo transformer / pseudo encoder.

- Se crea una layer de tipo self attention que sirve para imitar la atención de los transformadores
- Se imitan capas de feed forward a través de un Dense
- Se emplean conexiones residuales entre capas para realizar un aprendizaje más general
- Se emplean un monton de capas de tipo layer norm y dropout entre otras capas para normalizar los resultados y olvidar conexiones y que así nuestra IA aprenda más
- Se emplean bloques para combinar esto y será nuestra UNIDAD RECURSIVA, el equivalente a lo que sería un RNN anterior
- Lo que NO se usa son las capas de masking, los positional embeddings y las qkv cache por lo que no podemos definir esta estructura como un transformer Decoder.

Para hacernos una idea de como implementar el transformer seguí un ejemplo de transformer en torch hecho por andrej karpathy y ejemplos de transformers en keras

- <https://github.com/karpathy/minGPT>
- <https://www.tensorflow.org/text/tutorials/transformer?hl=es-419>
- https://keras.io/examples/nlp/text_classification_with_transformer/

3.c.d.a) Código

A continuación el código usado para implementar este modelo:

```
from keras import ops

class self_attn(keras.layers.Layer):
    """
```

```

Capa de self attention. Coge la layer de MultiheadAttention ya definida
en keras y la convierte en self attention haciendo que ambas entradas
sean los inputs
"""

def __init__(self, num_heads, embed_dim):
    super().__init__()
    self._mh_attn = tf.keras.layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embed_dim)

    def call(self, inputs):
        inputs = self._mh_attn(inputs, inputs)
        return inputs

class Block(keras.layers.Layer):
    """
    Bloque del pseudo-transformer
    - Coge la atencion de los inputs
    - la suma con los inputs (normaliza y dropea)
    - Coge el feed forward de los inputs (simplificado a una capa dense)
    - lo suma con los inputs (normaliza y dropea)
    """

    def __init__(self, num_heads, embed_dim, dropout):
        super().__init__()
        self._sa = self_attn(num_heads, embed_dim)

        self.ffw = layers.Dense(embed_dim)

        self.dropout1 = layers.Dropout(dropout)
        self.dropout2 = layers.Dropout(dropout)

        self._lyn1 = keras.layers.LayerNormalization(epsilon=1e-6)
        self._lyn2 = keras.layers.LayerNormalization(epsilon=1e-6)

    def call(self, inputs):
        attention = self._sa(inputs)
        attention = self.dropout1(attention)

        inputs = inputs + attention
        inputs = self._lyn1(inputs)

        ffw = self.ffw(inputs)
        ffw = self.dropout2(ffw)

        inputs = inputs + ffw

        inputs = self._lyn2(inputs)

        return inputs

class EncoderBlocks(keras.layers.Layer):
    """
    Capa recursiva, instancia units bloques seguidos para simular
    la recursividad de bloques
    """

```

```

def __init__(self, num_heads, embed_dim, dropout, units):
    super().__init__()
    self._blocks = [Block(num_heads, embed_dim, dropout) for _ in range(units)]

def call(self, inputs):
    for block in self._blocks:
        inputs = block(inputs)
    return inputs

class PseudoTransformer():
    """
    Traduce el modelo que estaba definido en el pdf a nuestro
    encoder definido a traves de las clases previas
    """
    def __init__(self,
                  max_tokens = 1000,
                  output_sequence_length = 75,
                  padding = False,
                  preprocessing = "lower_and_strip_punctuation",
                  output_dim = 32,
                  units = 8,
                  vocab = None,
                  dropout = 0.2,
                  num_heads = 2,
                  ) -> None:

        self._tokenizer_l = layers.TextVectorization(
            max_tokens=max_tokens,
            output_sequence_length= output_sequence_length,
            standardize = preprocessing,
            pad_to_max_tokens = padding
        )
        self._tokenizer_l.adapt(vocab)

        self._embedding_l = layers.Embedding(
            input_dim=max_tokens,
            output_dim=output_dim
        )

        self._blocks = EncoderBlocks(num_heads, output_dim, dropout, units)

        self._collapse_layer = layers.Dense(1, activation="sigmoid")

        self.model: keras.Sequential = keras.Sequential(
            [self._tokenizer_l, self._embedding_l, self._blocks, self._collapse_layer]
        )

        lr = PiecewiseConstantDecay([563*3, 563*7], [0.001, 0.0001, 0.00001])
        opt = AdamW(learning_rate=lr)
        self.model.compile(
            optimizer = opt, loss="binary_crossentropy", metrics=["accuracy"]
        )

    def summary(self):

```

```

    """
    Devuelve el summary de las capas del modelo
    """

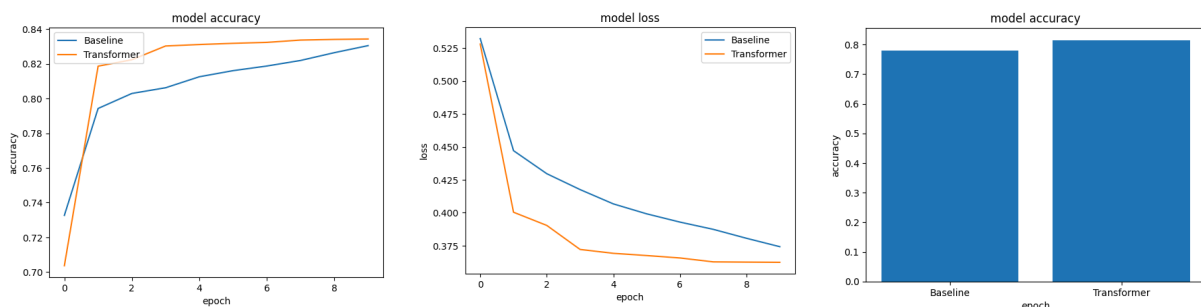
    return self.model.summary()

    def fit(self, X, Y, epochs = 10, batch_size = 128):
        """
        Hace el fitting del modelo en base a X e Y. Devuelve el histórico del
        entrenamiento
        """
        return self.model.fit(
            X, Y,
            epochs=epochs,
            batch_size=batch_size,
        )

    def test(self, X_test, Y_test):
        """
        Devuelve el accuracy del test
        """
        #devuelve [loss, acc, extras] entonces nos quedamos con el accuracy del
        test :D
        return self.model.evaluate(X_test, Y_test)[1]

```

3.c.d.b) Resultados



```

resultado_transformer = 0.8150197863578796

```

Podemos observar un test_accuracy de **0.815**. No es tan bueno como el mejor que teníamos de antes pero sigue siendo una mejora del **3.5%** respecto del baseline. Debido a las limitaciones computacionales es difícil sacarle el partido a esta estructura.

3.c.e) CONCLUSIÓN

La conclusión general del módulo de MLP es la siguiente:

- Combinando todos los mejores parámetros conseguimos **el mejor modelo**. Este presenta un test_accuracy del **0.848**. Comparado con el **0.78** del baseline supone casi un **7% de mejora frente al baseline**
- Realizando un bloque recursivo a mano obtenemos una mejora también, pero no tanta: Podemos observar un test_accuracy de **0.815**. No es tan bueno como el mejor que teníamos de antes pero sigue siendo una mejora del **3.5%** respecto del baseline. Debido a las limitaciones computacionales es difícil sacarle el partido a esta estructura.